

TECH_ARCHITECTURE.md

EdTech-платформа подготовки к ЕГЭ (профильная математика). Версия: 1.0 · Статус: утверждается · Автор: СТО / Software Architect

0. Резюме (TL;DR)

Рекомендуемый стек:

Слой	Выбор
Frontend + BFF	Next.js 15 (App Router) + React 19 + TypeScript
БД, Auth, Storage, Realtime	Supabase (PostgreSQL 16 + RLS + Auth + Storage)
ORM / миграции	Drizzle ORM + SQL-миграции Supabase
Хостинг приложения	Vercel (Hobby → Pro)
Кэш / rate limiting	Upstash Redis (serverless)
PDF	@react-pdf/renderer (этап 1) → Gotenberg/Playwright-сервис (этап 2)
Формулы	KaTeX (web) + MathJax → SVG (PDF)
Геометрия	Собственный Geometry Engine (параметрическая 3D-сцена → SVG-проекция), опционально Three.js для интерактива
Ошибки	Sentry
Логи	pino → Vercel Logs / Axiom
Продуктовая аналитика	PostHog (EU cloud)
CI/CD	GitHub Actions + Vercel Preview Deployments

Стартовая стоимость: **0–25 \$/мес** (Supabase Free + Vercel Hobby + Sentry Free). Рабочий продакшен с бэкапами и PITR: **~50–70 \$/мес** (Supabase Pro 25 \$ + Vercel Pro 20 \$ + Upstash ~0–10 \$).

Ключевые архитектурные решения:

- Одно Next.js-приложение** (монолит), разделённое на *feature-модули* — не микросервисы.
- Предметно-нейтральное ядро**: `exams` → `subjects` → `topics` → `prototypes`. ОГЭ/физика добавляются данными, а не кодом.
- Правильные ответы никогда не покидают сервер**. Проверка — только в Server Actions / Route Handlers с `service-role` ключом.

4. **Детерминированная генерация:** `generated_task = f(template, seed)` . Любое задание воспроизводимо.
5. **Organization-first:** `organization_id` есть в схеме с первого дня — B2B не потребует миграции модели.
-

1. Сравнение технических вариантов

1.1 Кандидаты

- **A. Next.js + Supabase (PostgreSQL)** — рекомендация.
- **B. Next.js + Firebase (Firestore + Auth + Storage + Cloud Functions).**
- **C. Next.js + Neon/Postgres + Auth.js + Drizzle + Cloudflare R2** («сборная» без BaaS).
- **D. Next.js + Convex.**
- **E. Next.js + PocketBase / Appwrite (self-hosted BaaS).**
- **F. Отдельный backend: NestJS/FastAPI + PostgreSQL + Docker (VPS).**

1.2 Матрица (1 — плохо, 5 — отлично)

Критерий	A Supabase	B Firebase	C Neon+Auth.js	D Convex	E PocketBase	F NestJS+VPS
Стоимость на старте	5	5	5	4	5	3
Стоимость при росте (10k+ учеников)	4	2 (чтения Firestore дорожают)	4	3	4	5
Сложность	4	4	3	4	4	2
Скорость разработки	5	4	3	5	4	2
Безопасность (RLS, изоляция данных)	5	3 (Security Rules сложны)	4	3	3	4
Масштабируемость	4	5	4	4	2	4
Удобство для Claude Code	5 (SQL, типы, стандартный стек)	3 (специфичные правила)	4	3 (свой язык запросов)	3	4
База заданий (реляционность, JSON, поиск)	5	2 (нет join, слабые запросы)	5	3	3	5
Авторизация	5	5	3 (сборка вручную)	4	3	3
Аналитика (SQL, matviews, витрины)	5	2 (только через BigQuery)	5	3	2	5

PDF	4 (Edge/Node functions, Storage)	4	4	3	3	5
Файловое хранилище	5 (S3-совместимое, RLS)	5	4 (R2 отдельно)	3	3	4
Будущая B2B-модель (multi-tenant)	5 (RLS по organization_id)	3	4	3	2	5
Vendor lock-in	2 (низкий — это Postgres)	5 (высокий)	1	4	2	1
Итого	63	51	56	49	43	53

1.3 Комментарии по кандидатам

A. Supabase. Обычный PostgreSQL + готовая Auth + Storage + Row Level Security. Всё, что ученик может видеть, описывается декларативно в SQL-политиках. База заданий — реляционная с `jsonb` для параметров, полнотекстовый поиск встроен. Экспорт — обычный `pg_dump`, уйти можно в любой момент. Минус: Free-план «засыпает» через неделю неактивности, PITR только на Pro.

B. Firebase. Отличная авторизация и масштабирование, но Firestore — документная БД без join. Статистика «средний процент по прототипу в группе за месяц» превращается в дорогие агрегации или выгрузку в BigQuery. Security Rules для teacher/student/group-изоляции громоздки и плохо тестируются. Ценообразование за чтения непредсказуемо на образовательных нагрузках (много мелких чтений).

C. Neon + Auth.js + Drizzle. Гибко и дёшево, но Auth, Storage, email-верификацию, сброс пароля, RLS-контекст собираем сами. Это +2–3 недели и постоянная поддержка. Рекомендуется как **план миграции** с Supabase, если понадобится: код на Drizzle переедет почти без изменений.

D. Convex. Очень быстрый DX, но свой язык запросов, слабая SQL-аналитика, дороже при росте, меньше знаний у моделей. Для задачного банка с тяжёлыми агрегациями — не оптимально.

E. PocketBase/Appwrite. Хороши для MVP-хобби, но SQLite (PocketBase) не тянет аналитику и многопользовательскую запись; нужен свой VPS и DevOps.

F. NestJS/FastAPI на VPS. Максимальный контроль и дешёвая масштабируемость, но требует DevOps (Docker, бэкапы, мониторинг, SSL, обновления). На старте это отвлекает от продукта. Разумный вариант на этапе >50k пользователей или для on-premise B2B-поставок.

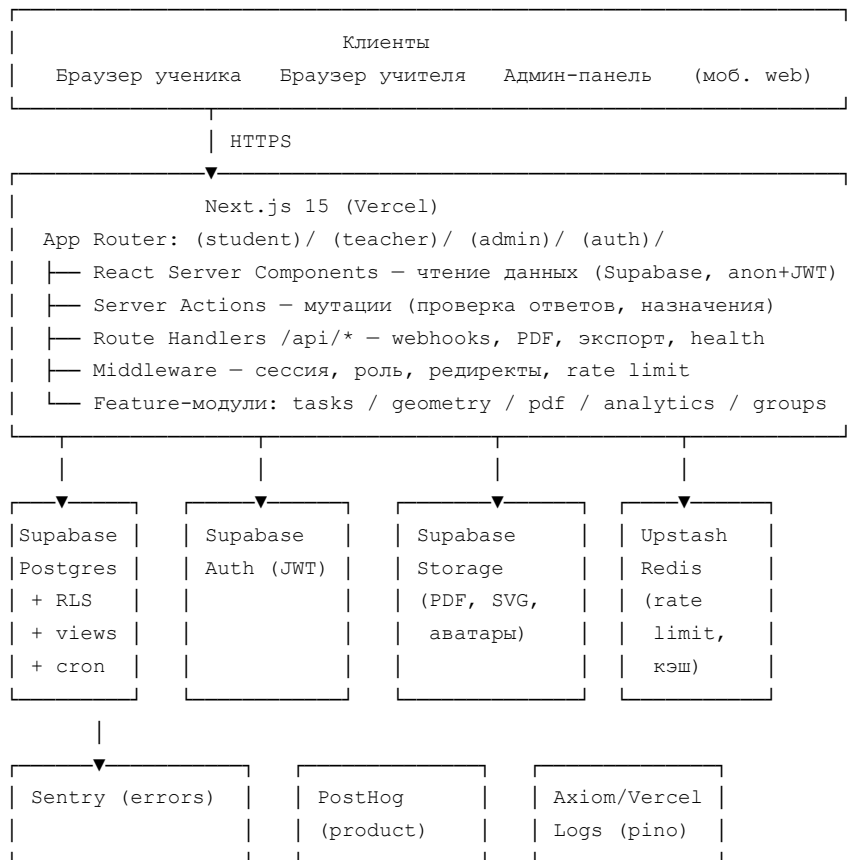
1.4 Рекомендация

Вариант A: Next.js + Supabase + Vercel, с ORM Drizzle и строгим правилом «вся бизнес-логика — в `src/features/*`, все запросы через репозитории». Это даёт:

- минимальную стоимость и минимальный DevOps;
- нативную безопасность через RLS;
- SQL-аналитику без внешних систем;
- стандартный стек, который Claude Code знает лучше всего;
- низкий lock-in: переезд на C или F — вопрос инфраструктуры, а не переписывания.

2. Архитектура приложения

2.1 Общая схема



2.2 Принципы

1. **Modular monolith.** Одно приложение, жёсткие границы между feature-модулями. Модуль экспортирует публичный `index.ts` ; импорт внутренностей чужого модуля запрещён (ESLint-правило `no-restricted-imports`).
2. **Server-first.** Чтение — в Server Components. Мутации — только через Server Actions с валидацией Zod. Клиентские компоненты — только там, где нужна интерактивность (ввод ответа, 3D-вью, таймер).
3. **Два ключа Supabase:**
 - `anon` + пользовательский JWT — всё, что делается от лица пользователя (RLS работает);
 - `service_role` — только на сервере, только в `src/lib/supabase/admin.ts` , только для: проверки ответов, генерации заданий, PDF, админ-операций. Никогда не импортируется в клиентский код (проверяется ESLint + `server-only`).
4. **Детерминизм.** Генерация задания — чистая функция `(template, seed) → task` . Это упрощает тесты, воспроизведение багов и генерацию PDF-вариантов.
5. **Предметная нейтральность ядра.** Ядро оперирует `exam / subject / topic / prototype` . Математика ЕГЭ — первый набор данных, а не `hard-code` .

2.3 Потоки данных (ключевые сценарии)

Ученик решает задание

1. Server Component загружает `generated_task` (без поля `correct_answer` — RLS + view `generated_tasks_public`).
2. Клиент рендерит условие (KaTeX) + чертёж (SVG из `figure_svg` или сгенерированный на лету).
3. Ученик вводит ответ → Server Action `submitAnswer(generatedTaskId, answer)`.
4. Сервер: rate limit → загружает `correct_answer` через service role → `checkAnswer()` (с допуском/нормализацией) → пишет `attempt` → при ошибке классифицирует в `errors` → возвращает результат.
5. Клиент показывает верно/неверно, разбор (если разрешён политикой задания).

Учитель создаёт вариант (worksheet)

1. Выбирает прототипы / шаблоны / кол-во вариантов.
 2. Server Action `createWorksheet()` → генерирует N вариантов детерминированно (`seed = hash(worksheetId, variantNo, position)`).
 3. Фоновая задача (Supabase `pg_cron` → Edge Function или Route Handler с QStash) рендерит PDF → Storage → запись в `generated_pdfs`.
 4. Учитель получает ссылку (signed URL, 1 час).
-

3. Frontend

- **Next.js 15 App Router**, React 19, TypeScript strict.
- **UI-кит**: shadcn/ui + Tailwind CSS 4. Иконки — lucide-react.
- **Формы**: react-hook-form + Zod (одна схема на клиент и сервер).
- **Состояние**: серверное — RSC; клиентское локальное — `useState` / `useReducer`; серверный кэш на клиенте — TanStack Query только для интерактивных экранов (таймер теста, live-статистика).
- **Формулы**: KaTeX (`react-katex`), формат условий — Markdown + `$...$`.
- **Чертежи**: SVG-компоненты из Geometry Engine; интерактивный 3D (react-three-fiber) — отдельный lazy-chunk, подключается только на страницах стереометрии.
- **Route groups**:

```
app/(auth)/login, register, reset
app/(student)/dashboard, practice, assignments, stats
app/(teacher)/dashboard, groups, worksheets, assignments, analytics
app/(admin)/prototypes, templates, users, organizations, system
app/api/...
```

- **Доступность и мобильность**: mobile-first, ученики часто заходят с телефона. Ввод ответа — числовая клавиатура (`inputmode="decimal"`).
 - **i18n**: заложен `next-intl`, локаль `ru` по умолчанию. Это дешево сейчас и дорого потом.
-

4. Backend

Backend = серверная часть Next.js + PostgreSQL-логика.

Тип операции	Где живёт
Чтение для UI	Server Components → <code>features/*/queries.ts</code>
Мутации	Server Actions → <code>features/*/actions.ts</code> (Zod-валидация, проверка прав, вызов service)
Бизнес-логика	<code>features/*/service.ts</code> (чистые функции, тестируемые без БД)
Доступ к данным	<code>features/*/repository.ts</code> (Drizzle)
Внешние webhooks, PDF, экспорт CSV	<code>app/api/*/route.ts</code>
Тяжёлые/фоновые задачи	Supabase Edge Functions или Route Handler + Upstash QStash (очередь)
Периодические задачи (агрегация статистики, очистка PDF)	<code>pg_cron</code> внутри Supabase
Инварианты данных	Postgres: constraints, triggers, RLS

Почему не отдельный API-сервер? На этапе до ~50k MAU Server Actions + RLS покрывают 100 % потребностей, при этом нет второго деплоя, второго CI и дублирования типов. Публичный REST/GraphQL для B2B-интеграций добавляется позже как `app/api/v1/*` с API-ключами организаций.

5. Database

PostgreSQL 16 (Supabase). Подробная схема — в `DATABASE_SCHEMA.md`.

Принципы:

- UUID v7 (упорядоченные) как первичные ключи.
- `created_at` / `updated_at` **везде**; `deleted_at` для soft-delete у пользовательских сущностей.
- `jsonb` для параметров шаблонов, сцен геометрии, результатов проверки — с JSON Schema валидацией на уровне приложения (Zod).
- RLS включён на **каждой** таблице. Политики — единственный источник истины по правам на данные.
- Представления `*_public` скрывают секретные колонки (`correct_answer`, `solution`).
- Материализованные представления для статистики, обновляются `pg_cron` каждые 5–15 минут.
- Миграции — только через файлы `supabase/migrations/*.sql`, никаких ручных правок в дашборде.

6. Authentication

Supabase Auth.

- Методы на старте: email + пароль, magic link. Позже: Google, Яндекс ID (OIDC), VK ID, Telegram Login.
- Сессии: cookie-based через `@supabase/ssr`; middleware обновляет токен.
- Регистрация ученика: по инвайт-коду группы или самостоятельно (тогда без группы — «самоподготовка»).
- Регистрация учителя: самостоятельно, но статус `pending` до подтверждения админом (защита от злоупотреблений на старте). Позже — верификация через организацию.
- Минимальный возраст и 152-ФЗ: согласие на обработку ПДн при регистрации, хранение согласия (`consents`). Для <14 лет — согласие представителя (флаг + email родителя, реализуется во 2-й итерации).
- Email-шаблоны Supabase кастомизируются; SMTP — Resend или Postmark.

7. Authorization

Три уровня:

1. **Route-level** — middleware проверяет наличие сессии и роль для route-групп (`student`), (`teacher`), (`admin`).
2. **Action-level** — каждый Server Action начинается с `const ctx = await requireRole('teacher')` и явной проверки владения ресурсом (`assertGroupOwner(ctx, groupId)`).
3. **Data-level** — RLS в Postgres. Даже если слой 1–2 сломаны, ученик физически не прочитает чужие данные.

Правило: **RLS — обязательный, слои 1–2 — для UX и ранних ошибок.** Подробно — `SECURITY.md`.

8. Roles

Роль	Описание	Где хранится
student	Решает задания, видит свою статистику	profiles.role + JWT claim app_role
teacher	Управляет группами, назначает задания, видит статистику своих учеников	то же
org_admin	(B2B) Управляет учителями и учениками организации	то же + organization_members.role
admin	Управляет контентом (прототипы, шаблоны), пользователями, системой	то же
content_editor	(позже) Только редактирование банка заданий	то же
parent	(позже) Только чтение статистики ребёнка	связь guardians

Роль попадает в JWT через `auth.hook_custom_access_token` (Supabase Auth Hook), чтобы RLS-политики читали `auth.jwt() ->> 'app_role'` без лишнего `join`.

9. Storage

Supabase Storage (S3-совместимый), бакеты:

Bucket	Публичный?	Содержимое	Политика
figures	да (CDN, immutable)	SVG/PNG чертежей прототипов и шаблонов	чтение всем, запись — admin
pdfs	нет	Сгенерированные варианты и решения	владелец worksheet / его организация; signed URL 1 ч
avatars	да	аватары	владелец
imports	нет	CSV-импорт учеников, загрузки админа	загрузивший + admin
exports	нет	CSV/XLSX-экспорты статистики	владелец, TTL 24 ч

Правила: файлы называются по `{entityType}/{entityId}/{hash}.{ext}` ; метаданные (размер, хэш, TTL) — в таблице `files` . Очистка просроченных — `pg_cron` . При росте объёма PDF (>100 GB) — вынос в Cloudflare R2 (без egress-платы), интерфейс `StorageProvider` в `src/lib/storage/` это допускает.

10. API

Внутренний API — Server Actions (типизированные, без ручного REST). Соглашения:

- Вход валидируется Zod-схемой из `features/*/schemas.ts` .
- Возврат — `Result<T, AppError> ( { ok: true, data } | { ok: false, error: { code, message } } )` , исключения наружу не летят.
- Идемпотентность мутаций через `idempotency_key` там, где возможен двойной сабмит (проверка ответа, создание worksheet).

Route Handlers (`app/api/`):

- `POST /api/webhooks/supabase` — auth-события.
- `GET /api/pdf/[id]` — отдача PDF по signed-проверке.
- `GET /api/health` — для мониторинга.
- `POST /api/jobs/*` — приёмники QStash (фоновые задачи), защищены подписью.

Публичный API v1 (этап B2B): `app/api/v1/*` , OpenAPI-спека генерируется из Zod (`zod-to-openapi`), авторизация по `X-API-Key` организации, `rate limit per-key`.

11. Task Engine

Ядро продукта. Живёт в `src/tasks/` (чистый TypeScript, без React и без Supabase — тестируется изолированно).

11.1 Модель

```
Exam (ЕГЭ проф.)
├─ Subject (математика)
│   └─ Topic (№3 стереометрия)
│       └─ Prototype (тип задачи из банка ФИПИ: «объём цилиндра по  $r$  и  $h$ »)
│           └─ TaskTemplate (конкретный параметрический генератор с чертежом-схемой)
│               └─ GeneratedTask (экземпляр: параметры, условие, ответ, seed)
│                   └─ Attempt (попытка ученика)
```

11.2 Контракт шаблона

```
interface TaskTemplateDefinition {
  code: string; // 'stereo.cylinder.volume.v1'
  prototypeCode: string; // 'ege.math.3.cylinder_volume'
  params: ParamSpec[]; // { name:'r', type:'int', min:1, max:12 } ...
  constraints: Constraint[]; // выражения, которым должны удовлетворять параметры
  derive: (p: Params) => Derived; // вычисляемые величины
  statement: Template; // Markdown+LaTeX с плейсхолдерами {{r}}
  answer: (p, d) => Answer; // { value: number, kind:'number'|'set'|'string', tolerance? }
  solution?: Template; // разбор
  figure?: FigureSpec; // ссылка на Geometry Engine
  difficulty: 1|2|3|4|5;
  tags: string[]; // 'cylinder','volume','pi'
  version: number;
}
```

- **Генерация:** `generate(def, seed) → GeneratedTask`. Seeded PRNG (mulberry32/xoshiro), ретраи до выполнения `constraints`, лимит попыток → ошибка «шаблон нереализуем».
- Ответы ЕГЭ — «красивые»: `constraints` гарантируют целые/конечные десятичные ответы там, где нужно.
- Проверка: `checkAnswer(expected, given)` нормализует запятую/точку, пробелы, знак, сравнивает с `tolerance`; для множеств — без учёта порядка.
- Шаблоны хранятся в БД (`task_templates.definition jsonb`) и имеют кодовые реализации в `src/tasks/templates/*.ts` для `derive/answer` (функции нельзя безопасно хранить в БД). БД-запись ссылается на `code`, реестр `templateRegistry` сопоставляет. Позже — DSL с безопасным интерпретатором (mathjs с ограниченной областью).
- **Версионирование:** изменение шаблона создаёт новую версию; старые `generated_tasks` хранят `template_version`.

11.3 Классификация ошибок

`errors` — таблица типовых ошибок ученика по прототипу (`error_types` : «перепутал радиус и диаметр», «забыл $1/3$ у пирамиды»). Классификатор — правило в шаблоне: список «дистракторов» с формулой неверного ответа. При совпадении ответа с дистрактором фиксируется `error_type_id`. Это основа персональной аналитики.

12. Geometry Engine

src/geometry/ — чистый TypeScript, без React.

12.1 Задачи

- Построить по параметрам сцену стереометрической фигуры (призма, пирамида, цилиндр, конус, шар, комбинации).
- Спроецировать 3D → 2D (кабинетная/аксонометрическая проекция с невидимыми линиями пунктиром — как в учебнике).
- Отрисовать подписи (буквы вершин, размеры, углы) без наложений.
- Выдать SVG (для web и PDF) и опционально 3D-модель для интерактива.
- Планиметрия (Nº1) — то же ядро, без проекции.

12.2 Слои

```
primitives/  Point3, Vector3, Segment, Polygon, Circle, Plane
solids/      prism(), pyramid(), cylinder(), cone(), sphere(), composite()
projection/  cabinet(), isometric(), hiddenLineDetection()
layout/      labelPlacement(), dimensionLines()
render/      toSVG(scene, opts), toThreeJSON(scene)  // react-three-fiber на фронте
spec/       FigureSpec (jsonb) — декларативное описание для хранения в БД
```

- SVG — детерминирован по FigureSpec + seed, кэшируется в figures -bucket по хэшу.
- Стиль — единый (толщина линий, шрифт Times/Liberation Serif для совпадения с бланками).
- Тесты — snapshot SVG + геометрические инварианты (например, число видимых рёбер).

13. PDF Engine

src/pdf/ .

Этап 1 (MVP): @react-pdf/renderer в Route Handler (Node runtime, не Edge).

- Формулы: MathJax (mathjax-full) → SVG на сервере → вставка как изображение/SVG.
- Чертежи: SVG от Geometry Engine напрямую.
- Шаблоны документов: «Вариант» (условия), «Ответы», «Решения», «Бланк ответов»; колонтитул с QR-кодом (worksheetId, variantNo) для будущей проверки по фото.
- Шрифты: Liberation Serif / PT Serif (кириллица, лицензия OFL).

Этап 2: при усложнении вёрстки — HTML/CSS → PDF через **Gotenberg** (Docker на Railway/Fly, ~5 \$/мес) или Playwright в отдельном сервисе. Интерфейс PdfRenderer позволяет заменить реализацию без изменения фич.

Асинхронность: генерация >2с уходит в очередь (QStash → /api/jobs/render-pdf), UI показывает статус pending → ready → failed через Supabase Realtime.

14. Analytics

Два контура:

1. Учебная аналитика (в Postgres):

- Источник истины — `attempts`.
- Витрины (materialized views): `mv_student_topic_stats`, `mv_group_prototype_stats`, `mv_error_heatmap`.
- Обновление `pg_cron` каждые 10 минут; для «моя статистика сейчас» — прямые запросы с индексами (объём на ученика мал).
- Метрики: точность по прототипу, среднее время, динамика за 7/30 дней, «слабые прототипы», прогноз балла (простая модель по весам прототипов ЕГЭ).
- Экспорт учителю: CSV/XLSX.

2. Продуктовая аналитика: PostHog (EU): воронки регистрации, удержание, feature-usage. События — через `src/analytics/track.ts`, единый словарь событий `events.ts`. ПДн в PostHog не отправляем (только `user_id`).

15. Teacher Cabinet

- **Группы:** создание, инвайт-код/ссылка, импорт учеников CSV, архивирование.
 - **Назначения (assignments):** выбрать прототипы/шаблоны, количество заданий, режим (тренировка / контроль), дедлайн, персональные варианты.
 - **Варианты (worksheets):** конструктор → N вариантов → PDF (условия / ответы / решения).
 - **Аналитика:** сводка по группе (heatmap ученик x прототип), детали ученика, типовые ошибки, дедлайны и невыполненные.
 - **Банк:** просмотр прототипов и шаблонов, «предпросмотр 5 случайных заданий».
-

16. Student Cabinet

- **Дашборд:** прогресс по темам, слабые прототипы, ближайшие дедлайны.
 - **Тренировка:** выбор темы/прототипа → бесконечная лента заданий; мгновенная проверка; разбор после ответа (или после N попыток — настраивается).
 - **Задания от учителя:** список, таймер (в режиме контроля), результат.
 - **Статистика:** графики точности и времени, история ошибок с объяснениями.
 - **Профиль:** цель по баллам, дата экзамена, уведомления.
-

17. Admin panel

- **Контент:** CRUD прототипов, шаблонов (с валидацией и «сгенерировать 100 и проверить constraints»), теги, сложность, версии, публикация.
- **Пользователи:** поиск, роли, блокировка, подтверждение учителей, слияние аккаунтов.

- **Организации (B2B):** создание, лимиты, тарифы, API-ключи.
- **Система:** очередь PDF, ошибки генерации, состояние cron, feature flags.
- Реализуется в том же Next.js (`app/(admin)`), UI — shadcn Data Table. Внешние админки (Retool/Refine) не нужны на старте.

18. Error tracking

- **Sentry** (`@sentry/nextjs`): клиент, сервер, edge; source maps на деплое; release = git SHA.
- Теги: `role`, `feature`, `template_code` для ошибок генерации.
- Алерты в Telegram/Slack при новых issue и всплесках.
- ПДн: `sendDefaultPii: false`, маскирование email в `beforeSend`.
- Доменные ошибки (`AppError`) с `expected: true` в Sentry не шлём.

19. Logging

- **pino** (JSON), обёртка `src/lib/logger.ts` с `requestId`, `userId` (хэш), `feature`.
- Уровни: `debug` локально, `info` prod.
- Vercel Logs (7 дней) → **Axiom** (бесплатно до 500 GB/мес) для поиска и дашбордов.
- Аудит-лог в БД (`audit_log`): изменения ролей, публикация шаблонов, удаление групп, выдача API-ключей — то, что должно пережить ротацию логов.
- Никогда не логируем: пароли, токены, полные ответы на задания вместе с `correct_answer`.

20. Backup strategy

Что	Как	Частота	Хранение
Postgres	Supabase daily backup (Pro)	ежедневно	7 дней
Postgres PITR	Supabase PITR add-on (~10 \$/мес)	непрерывно	7 дней
Postgres внешняя копия	GitHub Action <code>pg_dump</code> → Cloudflare R2 (шифрование age)	ежедневно	30 дней + 12 месячных
Storage (pdfs, figures)	<code>rclone sync</code> Supabase → R2	ежедневно	30 дней
Схема БД	миграции в git	каждый коммит	всегда
Банк заданий	<code>content/exports/*.json</code> в git через админ-экспорт	при публикации	всегда

- Ежеквартальное учение: восстановление из внешней копии на staging-проект Supabase, фиксируется в `docs/ops/restore-drills.md`.
- RPO ≤ 24 ч (Free) / ≤ 2 мин (PITR); RTO ≤ 2 ч.

21. Deployment

Окружения:

Env	Ветка	Vercel	Supabase
local	любая	next dev	supabase start (Docker)
preview	PR	Preview Deployment	staging-проект (или Supabase Branching)
staging	develop	Preview с фикс. алиасом	staging-проект
production	main	Production	prod-проект

Пайплайн (GitHub Actions):

1. lint → typecheck → test:unit → test:e2e (Playwright, smoke).
2. supabase db lint + проверка, что миграции применяются на чистой БД.
3. Merge в main → Vercel деплой → supabase db push через Action с секретом → Sentry release.
4. Миграции всегда **обратно совместимы** (expand/contract): сначала добавить, потом переключить код, потом удалить.

Домены: `app.<domain>`, `api.<domain>` (позже), `admin.<domain>` (тот же деплой, роутинг по хосту в middleware).

Секреты: Vercel Env Vars + GitHub Secrets; локально `.env.local` (в `.gitignore`), шаблон `.env.example`.

22. Дорожная карта архитектуры

Этап	Содержание	Инфра
М0 (2 нед.)	Скелет, Auth, роли, RLS, CI, Sentry	Free
М1 (4–6 нед.)	Task Engine + Geometry Engine для №3, тренировка ученика, 15–20 шаблонов	Free
М2 (3–4 нед.)	Кабинет учителя: группы, назначения, базовая статистика	Supabase Pro
М3 (2–3 нед.)	Worksheets + PDF, экспорт	+ QStash
М4	№1, №2, №4, №5 — только контент и новые генераторы	—
М5	№6 «Лаборатория» — отдельный feature-модуль с интерактивной геометрией	—
М6	B2B: организации, API v1, тарифы, SSO	+ R2

23. Риски и меры

Риск	Мера
Vendor lock-in Supabase	Drizzle + чистые SQL-миграции; ежедневный <code>pg_dump</code> наружу
Утечка ответов	Ответы только на сервере; RLS; тесты «ученик не может прочитать <code>correct_answer</code> » в CI
Сложность Geometry Engine	Начать с 5 тел и кабинетной проекции; интерактив — позже
Холодные старты Vercel для PDF	Очередь + статус; при росте — отдельный PDF-сервис
Рост стоимости Vercel (bandwidth)	Чертежи и PDF раздаются с Supabase CDN / R2, не через Next
Регуляtorика (152-ФЗ, локализация ПДн)	Согласия; при требовании — самохостинг Postgres в РФ (вариант F) без изменения кода